Machine Learning

Complete Revision Notes

AI/ML Engineering Revision Journey — Module 4 (Day 1 & Day 2)

Part 1: Supervised Learning — Regression, Classification, Model Evaluation, Bias-Variance, Cross-Validation
Part 2: Unsupervised Learning — Clustering (K-Means, Hierarchical, DBSCAN), Dimensionality Reduction (PCA)

Table of Contents

Introduction to Machine Learning

Machine Learning is the field of building systems that learn patterns from data instead of following hand-written rules. It splits into two core paradigms, based on whether the training data includes the correct answer or not.

	Supervised Learning	Unsupervised Learning
Data	Labeled (X, y)	Unlabeled (X only)
Goal	Predict a known label	Discover hidden structure
Example task	Predict pass / fail	Group similar customers
Example algorithms	Linear/Logistic Regression, Random Forest	K-Means, PCA, DBSCAN
Evaluation	Accuracy, F1, RMSE (vs ground truth)	Silhouette Score, inertia (no ground truth)

> These two notes sections mirror Module 4, Day 1 (Supervised Learning) and Day 2 (Unsupervised Learning) of the revision journey.

PART 1

Supervised Learning — Learning from Labeled Data

1. What is Supervised Learning?

In Supervised Learning, every training example is a pair: **(input features, correct output)**. The model learns a function that maps features → output by minimizing the error between its predictions and the true labels.

Term	Meaning
Input	Features (X) — e.g. study hours, attendance, age, pixel values
Output	Label / Target (y) — the correct answer we want to predict
Goal	Learn f(X) ≈ y so it generalizes to new, unseen data

2. Regression vs Classification

Type	Predicts	Example
Regression	A continuous number	House price, exam score, temperature
Classification	A category / class	Spam or not spam, disease or healthy, digit 0-9

3. The Supervised Learning Workflow

```
Labeled Data
   -> Train / Test Split
   -> Choose a Model
   -> Train (fit) the Model on the Training Set
   -> Predict on the Test Set
   -> Evaluate with Metrics
   -> Tune / Improve the Model
```

This same pipeline applies whether solving a regression or classification problem — only the model and evaluation metrics change.

4. Regression — Linear Regression

Linear Regression assumes a linear relationship between input and output: **score = w × study_hours + b**, and learns the best weight **w** and bias **b** by minimizing the Mean Squared Error between predictions and actual values.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Regression Evaluation Metrics

Metric	What it measures
MAE (Mean Absolute Error)	Average absolute difference between predicted and actual values
MSE (Mean Squared Error)	Average squared difference — penalizes larger errors more
RMSE (Root MSE)	Same units as the target, easier to interpret than MSE
R^2 Score	Proportion of variance in the target explained by the model (closer to 1 is better)

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)
```

5. Classification Algorithms

Common classic classification algorithms:

Algorithm	How it works
Logistic Regression	A linear model that predicts class probabilities
K-Nearest Neighbors (KNN)	Classifies a point based on its closest neighbors
Decision Tree	Splits data using a series of if/else rules
Random Forest	An ensemble of many decision trees (usually more accurate & robust)

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(max_depth=4),
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=5),
}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
```

6. Classification Evaluation Metrics

Metric	What it measures
Accuracy	Overall proportion of correct predictions
Precision	Of all predicted positives, how many were actually positive? (avoids false alarms)
Recall	Of all actual positives, how many did we correctly catch? (avoids missed cases)
F1 Score	Harmonic mean of Precision and Recall — a balanced single score

Accuracy alone can be misleading on imbalanced datasets — always check Precision, Recall, and F1 too.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
```

```
    confusion_matrix, roc_curve, roc_auc_score
)

accuracy_score(y_test, preds)
precision_score(y_test, preds)
recall_score(y_test, preds)
f1_score(y_test, preds)

cm = confusion_matrix(y_test, preds)

probs = model.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, _ = roc_curve(y_test, probs)
auc = roc_auc_score(y_test, probs)
```

The **Confusion Matrix** breaks predictions into True Positives, True Negatives, False Positives, and False Negatives. The **ROC Curve** and its **AUC** (Area Under Curve) show how well a model separates the two classes across all possible decision thresholds.

7. Bias-Variance Tradeoff (Overfitting vs Underfitting)

Concept	Description	Symptom
Underfitting (High Bias)	Model too simple to capture patterns	Poor performance on both train & test data
Overfitting (High Variance)	Model memorizes training data, including noise	Great on train data, poor on test data
Good Fit	Model captures the true underlying pattern	Consistent performance on train & test data

A classic diagnostic: compare training accuracy vs test accuracy as model complexity (e.g. tree depth) increases. If training accuracy keeps climbing while test accuracy plateaus or drops, the model is overfitting.

8. Cross-Validation

A single train/test split can be lucky or unlucky. **K-Fold Cross-Validation** splits the data into K folds, trains on K-1 folds and validates on the remaining one, repeating K times — giving a more reliable estimate of how the model performs on unseen data.

```
from sklearn.model_selection import cross_val_score, KFold

kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold, scoring="accuracy")

print("Mean CV Accuracy:", scores.mean(), "+/-", scores.std())
```

PART 2

Unsupervised Learning — Finding Structure Without Labels

9. What is Unsupervised Learning?

In Unsupervised Learning, we only have input features (X) — there is no target label (y). The model's goal is to find structure in the data itself:

- Which data points are similar and should be grouped together? (Clustering)
- Can we reduce the number of features while keeping the important information? (Dimensionality Reduction)
- Which data points look unusual or different from the rest? (Anomaly Detection)

10. Supervised vs Unsupervised — Quick Comparison

	Supervised Learning	Unsupervised Learning
Data	Labeled (X, y)	Unlabeled (X only)
Goal	Predict a known label	Discover hidden structure
Example task	Predict pass/fail	Group similar customers
Example algorithms	Linear/Logistic Regression, Random Forest	K-Means, PCA, DBSCAN
Evaluation	Accuracy, F1, RMSE (vs ground truth)	Silhouette Score, inertia (no ground truth)

11. Clustering — K-Means

K-Means groups data into K clusters by:

- Randomly placing K cluster centers ("centroids")
- Assigning every point to its nearest centroid
- Moving each centroid to the average position of its assigned points
- Repeating until the centroids stop moving

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

kmeans = KMeans(n_clusters=4, n_init=10, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
centroids = kmeans.cluster_centers_
```

12. Choosing K — Elbow Method & Silhouette Score

Method	How it works
Elbow Method	Plot inertia (within-cluster sum of squares) for different K; the "elbow" bend is a good candidate for K
Silhouette Score	Measures how similar a point is to its own cluster vs other clusters (ranges -1 to 1; higher is better)

```
from sklearn.metrics import silhouette_score

for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X_scaled)
    print(k, km.inertia_, silhouette_score(X_scaled, labels))
```

13. Hierarchical Clustering

Hierarchical Clustering builds a tree of clusters (a dendrogram) by iteratively merging the closest pairs of points/clusters. Unlike K-Means, you don't need to choose K upfront — you can "cut" the dendrogram at any height to get a chosen number of clusters.

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

linked = linkage(X_scaled, method="ward")
dendrogram(linked)

agglo = AgglomerativeClustering(n_clusters=4)
clusters = agglo.fit_predict(X_scaled)
```

14. DBSCAN (Density-Based Clustering)

DBSCAN groups points that are closely packed together (dense regions) and marks points in low-density regions as noise/outliers — unlike K-Means, it doesn't force every point into a cluster, and it doesn't require choosing K in advance.

Parameter	Meaning
eps	Maximum distance between two points to be considered neighbors
min_samples	Minimum number of points required to form a dense region

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
clusters = dbscan.fit_predict(X_scaled)
# clusters == -1 marks noise/outlier points
```

15. Dimensionality Reduction — PCA

Real datasets often have many features, which makes them hard to visualize and can slow down or confuse ML models (the "curse of dimensionality"). Principal Component Analysis (PCA) reduces the number of features while preserving as much of the original variance (information) as possible.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(pca.explained_variance_ratio_)   # variance retained per component
```

A common real-world pattern is to use PCA to reduce dimensionality first, then cluster the reduced data — this can make clustering faster and sometimes more accurate on high-dimensional data.

16. Real-World Applications

Technique	Real-World Use Case
K-Means / Hierarchical Clustering	Customer segmentation, market research
DBSCAN	Anomaly/fraud detection, geographic hotspot detection
PCA	Data compression, noise reduction, visualization of high-dimensional data
Clustering + PCA	Gene expression analysis, image compression, recommender systems
Supervised Classification	Spam detection, medical diagnosis, credit risk scoring
Supervised Regression	Price prediction, demand forecasting, risk estimation

Key Takeaways

Supervised Learning

- Learns from labeled data: features (X) mapped to known outputs (y).
- Regression predicts continuous numbers; Classification predicts categories.
- Common regression metrics: MAE, MSE, RMSE, R^2.
- Common classification metrics: Accuracy, Precision, Recall, F1, Confusion Matrix, ROC/AUC.
- Overfitting = great on training data, poor on new data. Underfitting = poor on both.
- Cross-Validation gives a more trustworthy performance estimate than a single train/test split.

Unsupervised Learning

- Finds structure in data without labels — the model isn't told the "right answer."
- K-Means partitions data into K clusters by iteratively updating centroids.
- The Elbow Method and Silhouette Score help choose a good value of K.
- Hierarchical Clustering builds a dendrogram and doesn't require choosing K upfront.
- DBSCAN finds clusters based on density and can automatically detect outliers/noise.
- PCA reduces the number of features while preserving as much variance as possible.
- There's no ground-truth accuracy — evaluation relies on internal metrics like silhouette score.

Module 4 complete: Supervised Learning (Day 1) + Unsupervised Learning (Day 2) — the two core paradigms of Machine Learning.